

ALGORITMA DAN STRUKTUR DATA
LATIHAN PRAKTIKUM 3 QUEUE(ANTRIAN)



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

LATIHAN

3. Implementasikan studi kasus simulasi tempat parkir dengan ketentuan sebagai berikut:
- Gunakan struktur data dengan elemen data berupa Nomor mobil.
 - Gunakan struktur data array.
 - Buatlah sebuah flowchart yang meliputi proses Mobil masuk. Mobil paling depan keluar. Mobil tengah keluar disertai proses penampungan mobil sementara, memasukkan mobil dari penampungan sementara dan menyusun tempat parkir kembali.

```
1. #include <stdio.h>
2. #define max 10
3.
4. typedef struct {
5.     int item[max];
6.     int front;
7.     int rear;
8.     int count;
9. } queue;
10.
11. void inisialisasi(queue *);
12. void tambah(queue *, int);
13. int hapus(queue *);
14. void tampil(queue *);
15. void kluartengah(queue *, int);
16.
17. int main() {
18.     queue q;
19.     int pilihan, nomor_mobil, mobil_keluar;
20.
21.     inisialisasi(&q);
22.
23.     do {
24.         printf("\n\nsimulasi parkir");
25.         printf("\n1. mobil masuk");
26.         printf("\n2. mobil keluar (depan)");
27.         printf("\n3. mobil keluar (tengah)");
28.         printf("\n4. selesai");
29.         printf("\nmasukkan pilihan (1-4): ");
30.         scanf("%d", &pilihan);
31.
32.         switch (pilihan) {
```

```

33.         case 1:
34.             if (q.count == max) {
35.                 printf("\nantrian penuh");
36.             } else {
37.                 printf("masukkan nomor mobil: ");
38.                 scanf("%d", &nomor_mobil);
39.                 tambah(&q, nomor_mobil);
40.                 printf("\nmobil %d masuk", nomor_mobil);
41.             }
42.             break;
43.
44.         case 2:
45.             if (q.count == 0) {
46.                 printf("\nantrian kosong");
47.             } else {
48.                 mobil_keluar = hapus(&q);
49.                 printf("\nmobil %d keluar dari depan.",
mobil_keluar);
50.             }
51.             break;
52.
53.         case 3:
54.             if (q.count == 0) {
55.                 printf("\nantrian kosong");
56.             } else {
57.                 printf("masukkan nomor mobil yang ingin
keluar: ");
58.                 scanf("%d", &nomor_mobil);
59.                 keluartengah(&q, nomor_mobil);
60.             }
61.             break;
62.
63.         case 4:
64.             printf("\nsimulasi selesai\n");
65.             break;
66.
67.         default:
68.             printf("\npilihan tidak valid, silakan pilih
1-4");
69.     }
70.

```

```

71.         if (pilihan != 4) {
72.             printf("\n\nkondisi parkir\n");
73.             tampil(&q);
74.         }
75.
76.     } while (pilihan != 4);
77.
78.     return 0;
79. }
80.
81. void inisialisasi(queue *q) {
82.     q->front = q->rear = -1;
83.     q->count = 0;
84. }
85.
86. void tambah(queue *q, int item) {
87.     if ((q->rear == max - 1 && q->front == 0) || (q->rear + 1
        == q->front)) {
88.         return;
89.     }
90.
91.     if (q->rear == max - 1)
92.         q->rear = 0;
93.     else
94.         q->rear++;
95.
96.     q->item[q->rear] = item;
97.     q->count++;
98.
99.     if (q->front == -1)
100.         q->front = 0;
101. }
102.
103. int hapus(queue *q) {
104.     int data;
105.
106.     if (q->front == -1) {
107.         return 0;
108.     } else {
109.         data = q->item[q->front];
110.         q->count--;

```

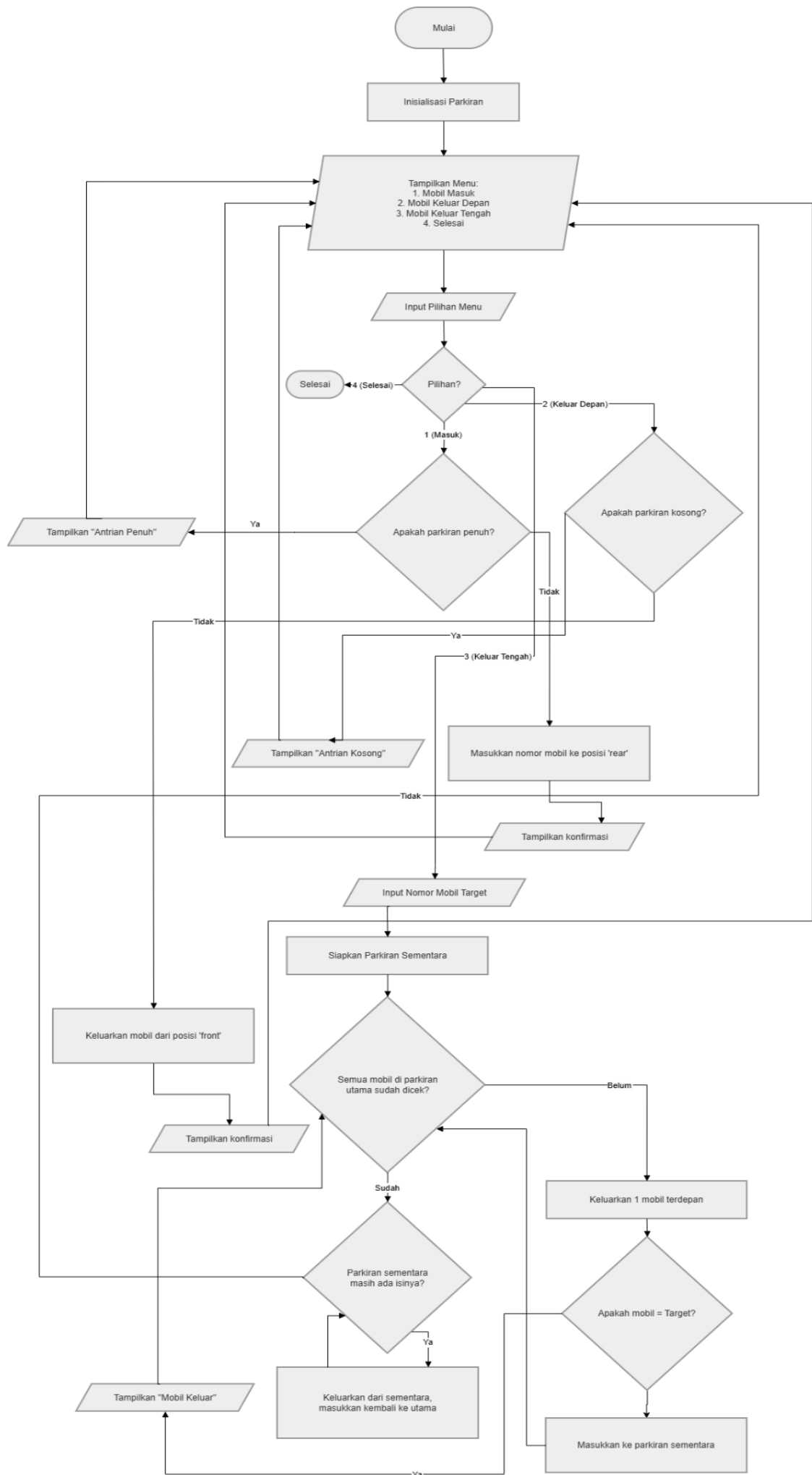
```

111.
112.         if (q->front == q->rear)
113.             q->front = q->rear = -1;
114.         else {
115.             if (q->front == max - 1)
116.                 q->front = 0;
117.             else
118.                 q->front++;
119.         }
120.     }
121.     return data;
122. }
123.
124. void tampil(queue *q) {
125.     if (q->count == 0) {
126.         printf("\nparkiran kosong");
127.         return;
128.     }
129.     int idx = q->front;
130.     for (int i = 0; i < q->count; i++) {
131.         printf("\nmobil : %d", q->item[idx]);
132.         idx = (idx + 1) % max;
133.     }
134. }
135.
136. void keluartengah(queue *q, int target_mobil) {
137.     queue temp;
138.     inisialisasi(&temp);
139.     int ketemu = 0;
140.     int jumlah_awal = q->count;
141.
142.     for (int i = 0; i < jumlah_awal; i++) {
143.         int mobil_keluar = hapus(q);
144.
145.         if (mobil_keluar == target_mobil) {
146.             printf("\nmobil %d keluar\n", target_mobil);
147.             ketemu = 1;
148.         } else {
149.             tambah(&temp, mobil_keluar);
150.         }
151.     }

```

```
152.
153.         if (!ketemu) {
154.             printf("\nmobil %d tidak ditemukan",
target_mobil);
155.         }
156.
157.         while (temp.count > 0) {
158.             int mobil_masuk = hapus(&temp);
159.             tambah(q, mobil_masuk);
160.         }
161.     }
```

Flowchart:



Output:

```
simulasi parkir
1. mobil masuk
2. mobil keluar (depan)
3. mobil keluar (tengah)
4. selesai
masukkan pilihan (1-4): 1
masukkan nomor mobil: 1

mobil 1 masuk

kondisi parkir
mobil : 1
```

```
kondisi parkir
mobil : 1
mobil : 2
mobil : 3

simulasi parkir
1. mobil masuk
2. mobil keluar (depan)
3. mobil keluar (tengah)
4. selesai
masukkan pilihan (1-4): 3
masukkan nomor mobil yang ingin keluar: 2

mobil 2 keluar

kondisi parkir
mobil : 1
mobil : 3

simulasi parkir
1. mobil masuk
2. mobil keluar (depan)
3. mobil keluar (tengah)
4. selesai
masukkan pilihan (1-4): █
```

```
kondisi parkir
mobil : 1
mobil : 2
mobil : 3
```

```
simulasi parkir
1. mobil masuk
2. mobil keluar (depan)
3. mobil keluar (tengah)
4. selesai
masukkan pilihan (1-4): 2

mobil 1 keluar dari depan.
```

```
kondisi parkir
mobil : 2
mobil : 3
```

```
simulasi parkir
1. mobil masuk
2. mobil keluar (depan)
3. mobil keluar (tengah)
4. selesai
masukkan pilihan (1-4): █
```

Analisis Program:

Pada program simulasi parkir yang menggunakan konsep antrean melingkar, eksekusi dimulai dengan memanggil fungsi inisialisasi untuk mengatur penunjuk front dan rear di angka -1 sebagai penanda bahwa area parkir masih benar-benar kosong. Ketika fungsi tambah dipanggil, program akan mengecek sisa kapasitas terlebih dahulu, lalu menggeser penunjuk rear secara memutar untuk menempatkan mobil baru di barisan paling belakang. Proses pemasukan ini sangat praktis karena mobil baru bisa langsung masuk tanpa perlu mengganggu urutan mobil lain yang sudah terparkir. Akhirnya, saat fungsi hapus dijalankan untuk mengeluarkan mobil, program cukup

memanggil mobil yang ditunjuk oleh front dan memajukan penunjuk tersebut ke indeks berikutnya. Dengan logika ini, ruang kosong yang ditinggalkan oleh mobil pertama tadi bisa dibiarkan saja, sehingga mobil-mobil di belakangnya tidak perlu repot bergeser maju satu per satu.

4. [Priority Queue] Terdapat struktur data mahasiswa yaitu nrp, nama dan nilai, masukkan satu persatu data mahasiswa dalam Queue, antrian di Queue berdasarkan prioritas nilai. Nilai yang tertinggi menjadi prioritas utama. Tanda asterisk (*) menyatakan pop/mengambil data dari Queue dan hasil dicetak di layar. Bagaimana output dari program ?

01 Andi 70	02 Budi 85	03 Candra 80	*	04 Didik 90	05 Edi 85	*	*	06 Fanda 75	*
------------------	------------------	--------------------	---	-------------------	-----------------	---	---	-------------------	---

```

1. #include <stdio.h>
2. #include <string.h>
3. #define max 10
4.
5. typedef struct {
6.     char nrp[5];
7.     char nama[50];
8.     int nilai;
9. } mahasiswa;
10.
11. typedef struct {
12.     mahasiswa item[max];
13.     int count;
14. } pqueue;
15.
16. void inisialisasi(pqueue *);
17. void tambah(pqueue *, char[], char[], int);
18. void hapus(pqueue *);
19. void tampil(pqueue *);
20.
21. int main() {
22.     pqueue q;
23.
24.     inisialisasi(&q);
25.
26.     tambah(&q, "01", "Andi", 70);
27.     tambah(&q, "02", "Budi", 85);
28.     tambah(&q, "03", "Candra", 80);

```

```

29.     hapus(&q);
30.
31.     tambah(&q, "04", "Didik", 90);
32.     tambah(&q, "05", "Edi", 85);
33.     hapus(&q);
34.     hapus(&q);
35.
36.     tambah(&q, "06", "Fanda", 75);
37.     hapus(&q);
38.
39.     puts("");
40.     return 0;
41.}
42.
43.void inisialisasi(pqueue *q) {
44.    q->count = 0;
45.}
46.
47.void tambah(pqueue *q, char nrp[], char nama[], int nilai) {
48.    if (q->count == max) {
49.        printf("\nantrian penuh");
50.        return;
51.    }
52.
53.    int i = q->count - 1;
54.
55.    while (i >= 0 && q->item[i].nilai < nilai) {
56.        q->item[i + 1] = q->item[i];
57.        i--;
58.    }
59.
60.    strcpy(q->item[i + 1].nrp, nrp);
61.    strcpy(q->item[i + 1].nama, nama);
62.    q->item[i + 1].nilai = nilai;
63.    q->count++;
64.}
65.
66.void hapus(pqueue *q) {
67.    if (q->count == 0) {
68.        printf("\nantrian kosong");
69.    } else {

```

```

70.         mahasiswa data = q->item[0];
71.         printf("\n* %s %s %d", data.nrp, data.nama,
            data.nilai);
72.
73.         for (int i = 0; i < q->count - 1; i++) {
74.             q->item[i] = q->item[i + 1];
75.         }
76.         q->count--;
77.     }
78. }
79.
80. void tampil(pqueue *q) {
81.     if (q->count == 0) {
82.         printf("\nantrian kosong");
83.         return;
84.     }
85.     for (int i = 0; i < q->count; i++) {
86.         printf("\nmahasiswa : %s %s %d", q->item[i].nrp, q-
            >item[i].nama, q->item[i].nilai);
87.     }
88. }

```

Output:

```

* 02 Budi 85
* 04 Didik 90
* 05 Edi 85
* 03 Candra 80

```

Analisis Program:

Sementara itu, pada program antrian mahasiswa yang menerapkan konsep antrian prioritas, proses diawali dengan pemanggilan fungsi inisialisasi yang sekadar mengatur jumlah data (count) menjadi 0. Hal ini dilakukan karena program tidak membutuhkan penunjuk front dan rear, mengingat mahasiswa yang nilainya paling tinggi sudah dipastikan akan selalu menempati indeks ke-0. Oleh karena itu, ketika fungsi tambah dipanggil, program harus bekerja lebih dinamis dengan cara mengecek dari barisan belakang dan menggeser mahasiswa bernilai lebih rendah agar mundur satu langkah. Pergeseran ini terus dilakukan sampai program

menemukan posisi yang tepat agar mahasiswa baru dapat menyisip sesuai dengan urutan prioritas nilainya. Terakhir, pada saat fungsi hapus dieksekusi, program akan mengeluarkan mahasiswa di indeks terdepan, yang mana proses ini mewajibkan seluruh sisa barisan di belakangnya untuk ikut bergeser maju satu langkah demi menutupi kekosongan di depan antrean tersebut.

Kesimpulan

Secara keseluruhan, kesimpulan dari kedua program di atas menunjukkan bahwa pemilihan struktur data sangat bergantung pada aturan main dari masalah di dunia nyata yang ingin diselesaikan.

Program simulasi parkir membuktikan bahwa pendekatan Circular Queue (Antrean Melingkar) sangat efektif untuk menjaga keadilan berdasarkan waktu kedatangan (FIFO) sekaligus menghemat penggunaan ruang memori, karena program hanya perlu menggeser penunjuk indeksnya saja tanpa harus memindahkan posisi fisik mobil di dalam antrean.

Sebaliknya, program antrean mahasiswa membuktikan bahwa pendekatan Priority Queue (Antrean Prioritas) lebih mutlak digunakan ketika nilai atau kriteria tertentu menjadi penentu utama dibandingkan waktu kedatangan, meskipun konsekuensinya program harus bekerja lebih keras dalam menggeser posisi data ke depan atau ke belakang setiap kali ada proses penambahan maupun penghapusan.

Pada akhirnya, kedua simulasi ini memberikan pelajaran penting bahwa tidak ada satu jenis antrean yang sempurna untuk semua kondisi; sistem parkir membutuhkan efisiensi pergerakan penunjuk waktu, sedangkan sistem penilaian membutuhkan ketegasan urutan prioritas data.